

Experimentelle Analyse von Algorithmen für fair-k-center mit Ausreißern

Bachelorarbeit

von

Carsten Krollmann

aus

Pirmasens

vorgelegt bei

Lehrstuhl für Algorithmen und Datenstrukturen

Prof. Dr. Melanie Schmidt

Heinrich-Heine-Universität Düsseldorf

Februar 2013

Betreuer:

Dr. Daniel Schmidt

Zusammenfassung

Hier beschreibe ich nun auf einer Seite, welches Problem ich betrachte, wie ich es gelöst habe und was die Hauptergebnisse meiner Arbeit sind.

Laut Prüfungsordnung muss es eine Zusammenfassung geben, die nicht länger als eine Seite ist.

Inhaltsverzeichnis

List of Algorithms	v
1 Einleitung	1
1.1 Motivation	1
1.2 Aufbau der Arbeit	1
2 Algorithmen	2
2.1 Matching	2
2.1.1 Bipatites Matching	2
2.2 Flussnetzwerke	3
2.2.1 Flusseigenschaften	3
2.2.2 Max-Flow Algorithmus	3
2.3 Gonzalez	4
2.4 Fairletts	5
2.5 Fair-Clustering mit Ausreißern	5
2.5.1 Outlier-Punkte	5
2.5.2 Red-Clustering	5
2.5.3 Fast-Anchor-Clustering	8
3 Aufbereiten der Daten	13
3.1 Ursprungsdaten	13
3.1.1 Problem der Datendokumentation	14
3.2 Cleaning-Pipeline	14
3.2.1 Normalisierung	14
3.2.2 Reinigen und Filtern	14
3.2.3 Subsampling	15
3.2.4 Format der Datensätze	15

4 Experimente	16
4.1 Graphen	16
4.1.1 Güte-Analyse	16
5 Auswertung	18
5.1 Beschreibung der Bilder	18
5.2 Güte der Algorithmen	18
5.3 Center-Aware Problem	18
6 Zitierungen	20
Literatur	21

List of Algorithms

1	Gonzalez-Algorithmus	4
2	Farilett-Finder Red Clustering	7
3	Red Clustering Algorihtmus	8
4	Farilett-Finder Fast-Anchor Clustering	10
5	Gonzalez-Algorithmus mit Fairletts	11
6	Fast-Anchor Clustering Algorihtmus	12

Kapitel 1

Einleitung

1.1 Motivation

Das hier ist die Einleitung. Hier sollte darauf eingegangen werden, was die Motivation der Arbeit ist.

1.2 Aufbau der Arbeit

Gehe kurz darauf ein, was den Leser in den nächsten Abschnitten erwartet. Die Gliederung kann vorab mit der betreuenden Person abgesprochen werden.

Kapitel 2

Algorithmen

In der Arbeit wurden einige verschiedene Algorithmen verwendet. Einige davon sind als gegeben vorausgesetzt. Diese sollen hier aufgezählt und erläutert werden

2.1 Matching

Ein Matching ist eine Aufteilung eines Graphen in disjunkte 2er Paare. Die Paare sollen sich dabei nicht überschneiden und die Knoten eines Paares sind immer über eine Kante verbunden sind. Sind alle Punkte in einem Paar spricht man von einem vollständigen Matching

2.1.1 Bipartites Matching

Wird ein Matching in einem Bipartiten Graphen, also einem Graphen welcher in zwei disjunkte Mengen aufgeteilt werden kann, sodass nur Kanten zwischen den zwei Mengen existieren, gesucht, dann ist das ein Bipartites Matching.

2.2 Flussnetzwerke

Das Bilden der Fairletts greift auf die Berechnung eines maximalen Flusses in einem Graph-Netzwerk und das daraus resultierende Matching zurück. Dabei haben die Kanten zwischen den Knoten eine Kapazität über welche ein Flusswert fließen kann. Zusätzlich gibt es noch zwei extra Knoten die Quelle s und die Senke t .

2.2.1 Flusseigenschaften

Für einen Fluss, welcher durch ein Flussnetzwerk fließt, müssen immer folgende 3 Eigenschaften gelten:

1. Jede Kante im Netzwerk muss eine nicht negative Kapazität und Fluss haben.
2. *Flusserhaltung*: Die Summe des einfließenden und des ausfließenden Flusses muss bei jedem Knoten mit Ausnahme von Quelle s und Senke t immer 0 sein. Bei der Quelle ist die Summe negativ, bei der Senke positiv.
3. *Kapazitätserhaltung*: Der Fluss welcher über eine Kante fließt, darf zu keinem Zeitpunkt die Kapazität dieser Kante überschreiten.

2.2.2 Max-Flow Algorithmus

Beim maximalen Fluss versucht man zwischen zwei Knoten, der Quelle s und der Senke t , möglichst viel Fluss zu schicken ohne die Flusseigenschaften zu verletzen. Im Folgenden wird das benutzt, um Knoten verschiedener Farbe in einem Bipatiten Graphen zu matchen. Bei der hier verwendeten Implementierung wurde auf die Bibliothek Lemon Library [Lem] zurückgegriffen. Diese bietet die Möglichkeit selbst Graphen aufzubauen und dann den Maximalen Fluss in diesen zu berechnen. Verwendet wurde hier der Preflow Algorithmus.

2.3 Gonzalez

Der Algorithmus von Gonzalez [Gon85] berechnet auf einer Punktmenge ein Clustering mit dem k-center Problem als Zielfunktion. Allerdings berücksichtigt er dabei keine Fairness. Dennoch wird er als Teil des fairen Algorithmus hier verwendet. Der Algorithmus ist ein greedy-Algorithmus, welcher k Cluster sucht. Er hat also als Eingabe eine Menge an Punkten mit einer Metrik (dist) und ein k welches die Anzahl an gewünschten Clustern angibt. Er beginnt damit, dass man einen beliebigen Punkt als Zentrum festlegt. Anschließend wird der Punkt, welcher am weitesten weg ist von diesem ersten Zentrum, als nächstes Zentrum gewählt. Für das dritte Zentrum wird der Punkt gewählt für den gilt, dass der Abstand zu den anderen Zentren maximal ist. Diese Schritte wiederholt man jetzt so oft bis man k verschiedene Zentren hat. Zum Schluss wird jeder Punkt dem Zentrum zugewiesen, zu welchem er den geringsten Abstand hat. Zurück gibt der Algorithmus die Zentren, die Knoten mit ihrer Clusterzugehörigkeit und den maximalen Radius, also der größte Abstand von einem Knoten zu seinem Zentrum.

Algorithm 1 Gonzalez-Algorithmus

```

1: function GONZALEZ( $P$ ,  $\text{dist}$ ,  $k$ )
2:    $Z :=$  leere Zentrenmenge
3:    $d_{\min} :=$  Distanz zu nächstem Zentrum
4:    $z_0$  beliebig aus  $P$ 
5:    $z_0.\text{cluster} = 0$ 
6:   for all  $x \in P$  do
7:      $d_{\min} = \text{dist}(z_0, x)$ 
8:      $x.\text{cluster} = 0$ 
9:   end for
10:  for  $i \in 2, \dots, k$  do
11:     $z_i = P(\text{argmax}(d_{\min}))$ 
12:     $z_i.\text{cluster} = i$ 
13:    for  $x \in P \setminus Z$  do
14:      if  $\text{dist}(z_i, x) < d_{\min}[x]$  then
15:         $d_{\min}[x] = \text{dist}(z_i, x)$ 
16:         $x.\text{cluster} = i$ 
17:      end if
18:    end for
19:  end for
20:  return  $(Z, P)$ 
21: end function

```

▷ in P wurden Punkte aktualisiert

2.4 Fairletts

In dieser Arbeit soll faires-Clustering analysiert werden, jedoch kann der Algorithmus von Gonzalez 1 eine solche Nebenbedingung nicht erfüllen. Wir benutzen hier sogenannte Fairletts. Fairletts sind die kleinstmögliche, faire Clustereinheit, welche die Nebenbedingung erfüllt. In dieser Analyse verwenden wir nur 1 : 1 Fairletts die aus einem roten und einem blauen Punkt bestehen. Die Grundidee ist, dass wir immer, wenn wir einen Punkt zu einem Cluster zuweisen, auch sein Fairlett-Partner dem selben Zentrum zugewiesen wird. So wird die Fairness nie verletzt [Fas23]. In dieser Arbeit werden zwei Arten der Fairlettbildung betrachtet und ihre Performance und Güte miteinander verglichen.

2.5 Fair-Clustering mit Ausreißern

2.5.1 Outlier-Punkte

In reellen Daten ist das Feature, welches man fair auf die Cluster verteilen will meist nicht perfekt 1 : 1 verteilt. Damit man trotzdem ein faires Clustering durchführen kann, ist es möglich einige Punkte als Outlier zu bestimmen. Diese Outlier werden dann nicht mehr beim Clustering berücksichtigt und werden dementsprechend auch nicht einem Cluster zugewiesen.

In einem Verhältniss von $n : n + x$ Punkten werden n 1 : 1 Fairletts gebildet und x Outlier definiert.

2.5.2 Red-Clustering

Als erste Art des Clusterings betrachten wir das Red-Clustering. Der hier verwendete Algorithmus ist eine Abwandlung des Algorithmus welcher im Paper [Chi+18] vorgestellt wurde für 1:1 Clustering. Hier wurde er erweitert um auch mit der Berücksichtigung von Ausreißern arbeiten zu können.

2.5.2.1 Bilden der Red-Cluster Fairletts 2

Erst gewährleisten wir, dass das kritische Feature, welches weniger, also nur n mal vorkommt, das rote Feature wird. Das gilt ohne Beschränkung der Allgemeinheit, da wir die Farbe linear einfach tauschen können.

Anschließend bauen wir einen bipartiten Graphen auf in dem die roten $R \in P$ und blauen $B \in P$ Punkte jeweils in eine Partition geteilt sind $R \cap B = \emptyset$, $R \cup B = P$ und $|R| \leq |B|$.

Als nächstes berechnen wir ein Matching mit einem Flussnetzwerk um die Fairletts zu bestimmen. Dazu fügen wir Quelle s und Senke t in das Netzwerk hinzu und ergänzen Kanten zwischen Quelle und den roten Knoten bzw. Kanten zwischen blauen Knoten und Senke, welche alle die Kapazität 1 haben. Nun brauchen wir noch die Kanten zwischen den roten und den blauen Punkten dazu "raten" wir ein r_{test} und fügen alle Kanten zwischen $a \in R$ und $b \in B$ für welche gilt $dist(a, b) \leq r_{test}$. Dabei wählen wir das r_{test} aus der Liste von allen möglichen paarweisen Distanzen $r_{test} \in \{dist(a, b) | a \in R, b \in B\}$.

In diesem Netzwerk wird nun der maximale Fluss berechnet und wenn der Flusswert f gleich der Anzahl an roten Punkten ist $f = |R|$, wissen wir, dass alle roten Punkte gematcht wurden. Diese Berechnung führen wir so lange mit verschiedenen $r_{test} \in \{dist(a, b)\}$ bis wir das kleinste r_{test} gefunden haben für die das Matching erfolgreich war ($f = |R|$). Um die Anzahl der Vergleiche zu verringern sortieren wir die Menge $\{dist(a, b) | a \in R, b \in B\}$ aufsteigend und suchen r_{opt} über die Binäre-Suche.

Um jetzt die Fairletts zueinander zu zuweisen erzeugen wir nochmal das Flussnetzwerk mit den Kanten $e = (a, b)$ für $\{dist(a, b) \leq r_{opt} | a \in R, b \in B\}$. Dann betrachten wir die Kanten zwischen den Partitionen R und B und sobald ein Fluss von 1 über die Kante e fließt, weisen wir a und b mit $e = (a, b)$ die selbe FairlettID zu, was sie zu einem Fairlett macht.

2.5.2.2 Finden der Outlier

Um die Ausreißer zu finden müssen wir noch alle blauen Punkte suchen, welche nicht gematcht wurden. Für jeden blauen Punkt b welcher nicht gematcht wurde gilt, dass der Fluss über die Kante $e_{t_i} = (b, t)$ 0 ist. Also gehen wir durch alle Senke-Kanten $e_{t_i} = (b_i, t)$ für alle $i \in [0, |B|]$ und wenn der Flusswert dieser Kante e_{t_i} 0 ist, dann markieren wir b als Ausreißer. Dieser Punkt wird bei der Zuweisung zu den Zentren ausgelassen

Algorithm 2 Farilett-Finder Red Clustering

```

1: function RED-CLUSTER-FAIRLETT( $P$ ,  $\text{dist}$ )
2:    $R := \{x \mid x \in P \wedge x.\text{color} = \text{RED}\}$ 
3:    $B := \{x \mid x \in P \wedge x.\text{color} = \text{BLUE}\}$ 
4:    $R_{\text{pot}} = \{r = \text{dist}(a, b) \mid a \in R \mid b \in B\}$ 
5:    $G :=$  Graph für Flussnetzwerk
6:   for all  $r \in R_{\text{pot}}$  do ▷ verwende Binäre Suche
7:     Füge in  $G$  alle Punkte aus  $R$  ein
8:     Füge in  $G$  alle Punkte aus  $B$  ein
9:     Füge in  $G$  Quelle  $s$  und Senke  $t$  ein
10:    Alle Kanten  $(s, a)$  und  $(b, t)$  mit Kapazität 1 einfügen
11:    for all  $\{(a, b) \mid a \in R \mid b \in B\}$  do
12:      if  $\text{dist}(a, b) \leq r$  then
13:        Füge Kante  $(a, b)$  in  $G$  mit Kapazität 1 ein
14:      end if
15:    end for
16:    Berechne Maximalen Fluss  $f$  von  $G$ 
17:    Vergleiche mit  $f \geq |R|$  für Binäre-Suche
18:  end for
19:  Baue  $G$  neu auf mit  $r_{\text{opt}}$  wie in der oberen Schleife auf
20:  Berechne den maximalen Fluss  $f$  auf  $G$ 
21:  for all  $e_{\text{main}_i} \in \{(a, b) \mid a \in R \mid b \in B\}$  do
22:    if  $f(e_{\text{main}_i}) = 1$  then
23:      Weise  $e.a$  und  $e.b$  dem gleichen Fairlett zu
24:    end if
25:  end for
26:  for all  $e_t \in \{(b, t) \mid b \in B\}$  do
27:    if  $f(e_t) = 0$  then
28:      Markiere  $b$  als Ausreißer
29:    end if
30:  end for
31: end function

```

2.5.2.3 Berechnen der roten Zentren 3

Jetzt Clustern wir lediglich die roten Punkten mithilfe des Algorithmus von Gonzalez 1. Dabei können wir sicher sein, dass dabei kein Ausreißer als Zentrum gewählt wird, da aufgrund der Bedingung am Anfang $|R| \leq |B|$ kein roter Punkt ein Ausreißer ist.

Nach diesem Schritt sind alle roten Punkten einem Cluster zugewiesen.

2.5.2.4 Zuweisen der blauen Partner

Zum Schluss gehen wir durch alle blauen Punkte durch und weisen sie dem selben Cluster zu, zu welchem auch ihr roter Fairlett-Partner zugewiesen wurde oder überspringen sie, falls sie Ausreißer sind.

Algorithm 3 Red Clustering Algorithmus

```

1: function RED-CLUSTERING( $P$ ,  $\text{dist}$ ,  $k$ )
2:   Berechne und markiere Fairletts mit RED-CLUSTER-FAIRLETT( $P$ ,  $\text{dist}$ )
3:    $R := \{x | x \in P \wedge x.\text{color} = \text{RED}\}$ 
4:   Berechne Clustering von  $R$  mit GONZALEZ( $R$ ,  $\text{dist}$ ,  $k$ )
5:    $B := \{x | x \in P \wedge x.\text{color} = \text{BLUE}\}$ 
6:   for all  $b \in B$  do
7:     weise  $b$  dem Selben Cluster zu wie sein Fairlett Partner
8:   end for
9: end function

```

2.5.3 Fast-Anchor-Clustering

Als zweiten Clustering-Algorithmus betrachten wir den Fast-Anchor-Algorithmus [Fas23]. Dieser arbeitet ähnlich wie der Red-Clustering Algorithmus 2.5.2 bildet aber nach einer grundlegend anderen Bedingung die Fairletts

2.5.3.1 Bilden der Anchor-Fairletts 4

Auch wie im Red-Clustering Algorithmus 2.5.2 bauen wir ein Flussnetzwerk zum finden der Fairletts auf. Unterschiedlich ist nur die Bedingung für das hinzufügen der Kanten zwischen $a \in R$ und

$b \in B$. Man testet für alle Paare $(a, b) \in \{a \in R | b \in B\}$ jeden Punkt $\alpha \in P$ ob α als Anker in Frage kommt. Dabei wird der Anker so gewählt, das $\max(\text{dist}(\alpha, a), \text{dist}(\alpha, b))$ minimal ist.

Diese Anker werden auch abgespeichert, denn durch sie bestimmen wir später welchem Cluster wir das Fairlett zuweisen.

Beim hinzufügen der Kanten zwischen $a \in R$ und $b \in B$ prüfen wir jetzt nicht mehr auf die Bedingung $\text{dist}(a, b) \leq r_{\text{test}}$, sondern betrachten den Anker also $\max(\text{dist}(\alpha, a), \text{dist}(\alpha, b)) \leq r_{\text{test}}$ und fügen dementsprechend Kanten mit Kapazität 1 ein.

Die Ausreißer werden genau so wie beim Red-Clustering bestimmt 2.5.2.2

2.5.3.2 Berechnen der Zentren

Wie bereits beim Red-Clustering 2.5.2 verwenden wir hier wieder den Algorithmus von Gonzalez allerdings wird dieser noch um eine Kleinigkeit erweitert. Beim normalen Gonzalez könnte es vorkommen, das zwei Punkte eines Fairletts beide als Zentrum gewählt werden. Da sie aber ein Fairlett sind, müssen sie beide dem gleichen Cluster angehören. Daher muss man immer wenn man einen Punkt als Zentrum wählt, seinen Partner als potenzielles Zentrum entfernen. Wir benutzen hier also einen erweiterten Gonzalez-Algorithmus 5 um das eigentliche Clustering vorzunehmen. Diesmal Clustern wir alle Punkte welche keine Ausreißer sind, insbesondere auch blaue Punkte. Die Zentren die in diesem Schritt berechnet werden sind auch immer gültig, da das Clustering ohne die Ausreißer berechnet wird.

2.5.3.3 Zuweisen der Punkte zu den Clustern

Zum Schluss müssen noch alle Punkte, welche keine Ausreißer sind einem Cluster hinzugefügt werden. Dazu geht man jetzt alle Punkte $p \in P$ durch und betrachtet ihren Anker α_p . Man weißt dann jeden Punkt p dem Zentrum $c \in Z$ zu, wobei Z die Menge der von Gonzalez 5 berechneten Zentren ist, wenn der Anker α_p diesem Zentrum am nächsten ist. Also $p.\text{cluster} = c.\text{cluster}$ wenn für c gilt $\min(\text{dist}(\alpha_p, c))$ für alle $c \in Z$

Algorithm 4 Farilett-Finder Fast-Anchor Clustering

```

1: function FAST-ANCHOR-FAIRLETT( $P$ ,  $\text{dist}$ )
2:    $R := \{x | x \in P \wedge x.\text{color} = \text{RED}\}$ 
3:    $B := \{x | x \in P \wedge x.\text{color} = \text{BLUE}\}$ 
4:    $R_{\text{pot}} = \{r = \text{dist}(a, b) | a \in R | b \in B\}$ 
5:    $G :=$  Graph für Flussnetzwerk
6:    $A :=$  Ankermatrix  $|R| \times |B|$ 
7:
8:   for all  $(a, b) | a \in R | b \in B$  do                                     ▷ Ankerberechnung
9:     for all  $\alpha \in P$  do
10:      if  $\max(\text{dist}(\alpha, a), \text{dist}(\alpha, b)) \leq A[a, b]$  then
11:         $A[a, b] = \max(\text{dist}(\alpha, a), \text{dist}(\alpha, b))$ 
12:      end if
13:    end for
14:  end for
15:
16:  for all  $r \in R_{\text{pot}}$  do                                               ▷ verwende Binäre Suche
17:    Füge in  $G$  alle Punkte aus  $R$  ein
18:    Füge in  $G$  alle Punkte aus  $B$  ein
19:    Füge in  $G$  Quelle  $s$  und Senke  $t$  ein
20:    Alle Kanten  $(s, a)$  und  $(b, t)$  mit Kapazität 1 einfügen
21:    for all  $\{(a, b) | a \in R | b \in B\}$  do
22:      if  $\max(\text{dist}(A[a, b], a), \text{dist}(A[a, b], b)) \leq r$  then
23:        Füge Kante  $(a, b)$  in  $G$  mit Kapazität 1 ein
24:      end if
25:    end for
26:    Berechne Maximalen Fluss  $f$  von  $G$ 
27:    Vergleiche mit  $f \geq |R|$  für Binäre-Suche
28:  end for
29:
30:  Baue  $G$  neu auf mit  $r_{\text{opt}}$  wie in der oberen Schleife auf
31:  Berechne den maximalen Fluss  $f$  auf  $G$ 
32:  for all  $e_{\text{main}_i} \in \{(a, b) | a \in R | b \in B\}$  do
33:    if  $f(e_{\text{main}_i}) = 1$  then
34:      Weise  $e.a$  und  $e.b$  dem gleichen Fairlett zu
35:    end if
36:  end for
37:  for all  $e_t \in \{(b, t) | b \in B\}$  do
38:    if  $f(e_t) = 0$  then
39:      Markiere  $b$  als Ausreißer
40:    end if
41:  end for
42: end function

```

Algorithm 5 Gonzalez-Algorithmus mit Fairletts

```

1: function GONZALEZ-FAIRLETT-AWARE( $P$ ,  $dist$ ,  $k$ )
2:    $Z :=$  leere Zentrenmenge
3:    $d_{min} :=$  Distanz zu nächstem Zentrum
4:    $z_0$  beliebig aus  $P$ 
5:    $z_0.cluster = 0$ 
6:    $z_0.partner.cluster = 0$  ▷ Fairlett-Awareness
7:   Entferne  $z_0.partner$  als potenzielles Zentrum aus  $P$ 
8:
9:   for all  $x \in P$  do
10:      $d_{min} = dist(z_0, x)$ 
11:      $x.cluster = 0$ 
12:   end for
13:
14:   for  $i \in 2, \dots, k$  do
15:      $z_i = P(argmax(d_{min}))$ 
16:      $z_i.cluster = i$ 
17:      $z_i.partner.cluster = i$  ▷ Fairlett-Awareness
18:     Entferne  $z_i.partner$  als potenzielles Zentrum aus  $P$ 
19:     for  $x \in P \setminus Z$  do
20:       if  $dist(z_i, x) < d_{min}[x]$  then
21:          $d_{min}[x] = dist(z_i, x)$ 
22:          $x.cluster = i$ 
23:       end if
24:     end for
25:   end for
26:   return  $(Z, P)$  ▷ in  $P$  wurden Punkte aktualisiert
27: end function

```

Algorithm 6 Fast-Anchor Clustering Algorihtmus

```

1: function FAST-ANCHOR-CLUSTERING( $P$ ,  $\text{dist}$ ,  $k$ )
2:   Berechne und markiere Fairletts mit FAST-ANCHOR-FAIRLETT( $P$ ,  $\text{dist}$ )
3:    $F := \{x | x \in P \wedge x \text{ ist kein Ausreißer}\}$ 
4:   Berechne Clustering von  $F$  mit GONZALEZ-FAIRLETT-AWARE( $R$ ,  $\text{dist}$ ,  $k$ )
5:    $\rightarrow$  gibt Zentrenmenge  $Z$  zurück
6:
7:   for all  $p \in P$  enumerate  $i$  do            $\triangleright$  Zentren und Partner werden sich selbst zugewiesen
8:     if  $p$  ist Zentrum then
9:        $p.\text{cluster} = i$ 
10:       $p.\text{partner.cluster} = i$ 
11:    end if
12:  end for
13:
14:   $U := \{x | x \in P \wedge x \text{ ist kein Zentrum, kein Ausreißer und kein Partner eines Zentrums}\}$ 
15:  for all  $p \in U$  do
16:     $\alpha := p.\text{anchor}$ 
17:    suche  $c \in Z$  sodass  $\text{dist}(\alpha, c)$  minimal
18:     $p.\text{cluster} = c.\text{cluster}$ 
19:  end for
20: end function

```

2.5.3.4 Center-Aware Zuweisung

Aufgrund dessen, dass wir die Punkte dem Zentrum zuweisen welches am nächsten am Ankerpunkt α ist könnte es sein das ein Zentrum nicht Teil seines eigenen Clusters ist. Nehmen wir an, ein Punkt $c \in Z$ wird als Zentrum gewählt, dessen Anker α ist. Zusätzlich gibt es noch ein weiteres Zentrum $c_2 \in Z$ und es gilt $\text{dist}(c, \alpha) > \text{dist}(c_2, \alpha)$. Dann würde bei der Zuweisung in 6 der Punkt c dem Cluster von Punkt c_2 zugewiesen werden. Um das zu verhindern gehen wir durch alle Punkte durch und wenn sie ein Zentrum sind, sollen sie ihrem eigenen Cluster zugewiesen werden. Das gleiche machen wir mit den Fairlett-Partnern der Zentren.

Dies kann in Sonderfällen aber zu einer Verschlechterung des maximalen Radius führen, obwohl man mehr Cluster zulässt. Dazu mehr im Kapitel Auswertung 5.3

Kapitel 3

Aufbereiten der Daten

Um vergleichbare Werte zu erhalten, haben wir uns an einem Paper orientiert, welches ebenfalls probiert hat, Fair-k-Center zu lösen [Chi+18]. Für die Vergleichbarkeit haben wir auch die gleichen Daten verwendet.

3.1 Ursprungsdaten

In dem Chierichetti-Paper [Chi+18] wurden 3 unterschiedliche Datensätze verwendet. Diese sind:

- Daten eines US-Zensus [BK96]
- Daten von Telefongesprächen einer Bank [MRC12]
- Einen Diabetes-Datensatz, wobei der im Paper [Chi+18] verlinkte Datensatz [Kah] nicht der richtige Datensatz sein kann (Er enthält andere Felder als die im Paper genannten). Nachforschungen haben gezeigt, dass sie in Wirklichkeit einen anderen Diabetes-Datensatz [Clo+14] benutzt haben.

3.1.1 Problem der Datendokumentation

Ein Problem der Daten war, dass sie einerseits nicht in einem normalisierten CSV Format vorhanden waren und andererseits die Autoren im Paper [Chi+18] Subsamples benutzt, jedoch nicht explizit gesagt haben, welche Datenpunkte genommen und welche Features der Daten verwendet wurden. Bei Letzterem wurden für jeden Datensatz ein paar Features genannt. Genau diese haben wir dann auch verwendet. Das Problem der Normalisierung und des Subsamplings sind wir dann selbst angegangen.

3.2 Cleaning-Pipeline

Um die Daten verwendbar zu machen, brauchen wir einige Schritte. Dafür verwenden wir ein Python-Skript, welches einerseits vollautomatisch, andererseits aber auch deterministisch und konfigurierbar die Daten aufbereitet. Auch werden die Daten nach der Normalisierung, der Reinigung und dem Zerteilen jeweils abgespeichert, damit man ggf. mit diesen Daten auch weiterarbeiten kann, ohne sie nochmal neu erzeugen zu müssen.

3.2.1 Normalisierung

Als erstes mussten die Daten in ein einheitliches CSV-Format übertragen werden. Das war zwar bei dem Diabetes-Datensatz [Clo+14] nicht notwendig, bei den anderen zwei [BK96] [MRC12] mussten ein paar Zeichen ersetzt werden und die Zensus-Daten brauchten Überschriften.

3.2.2 Reinigen und Filtern

Anschließend werden die Daten für unser Clustering weiterverarbeitet. Dazu werden alle Spalten entfernt, die wir nicht brauchen, sodass jeder Datensatz nur aus einigen numerischen Werten und dem Critical-Feature besteht. Gleichzeitig wird auch eine Grundanalyse der Critical-Features (Verhältnisse und mergen von Kategorien die das gleiche bedeuten) durchgeführt. Das Critical-Feature steht immer in der ersten Spalte. Ganz zum Schluss werden alle Daten auf ein Intervall von [-

1, 1] gemappt. So haben Daten mit unterschiedlichen Skalierungen nicht unterschiedlichen große Auswirkung auf das Clustering bzw. die berechneten Distanzen.

3.2.3 Subsampling

Da es um Faires-Clustering geht, sind die Versuche stark abhängig von der Verteilung der Ursprungsdaten. Um zu vermeiden, ein sehr ungünstiges Batch zu bekommen, werden alle Daten in 20 disjunkte Subsamples zerteilt, welche die selbe Größe haben wie sie im Paper [Chi+18] genannt werden. In der Analyse werden dann alle Subsamples getestet und dann der Durchschnitt über die Performance gebildet.

3.2.4 Format der Datensätze

Hier sollen jetzt nochmal klar die Datensätze dokumentiert werden. Von jedem Datensatz wurden jeweils 20 Disjunkte Subsamples erzeugt.

Tabelle 3.1: Datenformat

Datensatz	Critical-Feature	Verhältnis des Critical-Feature	Verwendete numerische Feature	Sub-sample-Größe
Zensusdaten [BK96]	sex (male = 0; female = 1)	Male 21790 / Female 10771 ≈ 2.02	age, fnlwtg, education-num, capital-gain, hours-perweek	600
Bankdaten [MRC12]	bin-marital (selbsterzeugt aus Spalte "marital") (single = 0; divorced = 0; married = 1)	es gibt single, married und divorced. Zähle single und divorced zusammen. Verhältnis: 27214 married / 17997 not-married ≈ 1.51	age, balance, duration of call	1000
Diabetesdaten [Clo+14]	sex (male = 0; female = 1)	Verhältnis Male 47055 / Female 54708 ≈ 0.86	age, time-in-hospital	1000

Kapitel 4

Experimente

- Was genau hab ich jetzt berechnet
- Welche Daten habe ich explizit verwendet (kurz)
- Diagramm der Testing-Pipeline
- Umgebung (vielleicht technische Daten)
- Graphen aus Python

4.1 Graphen

4.1.1 Güte-Analyse

Um die Algorithmen zu vergleichen betrachten wir ihre Güte auf den Test-Daten mit verschiedener Anzahl an Clustern

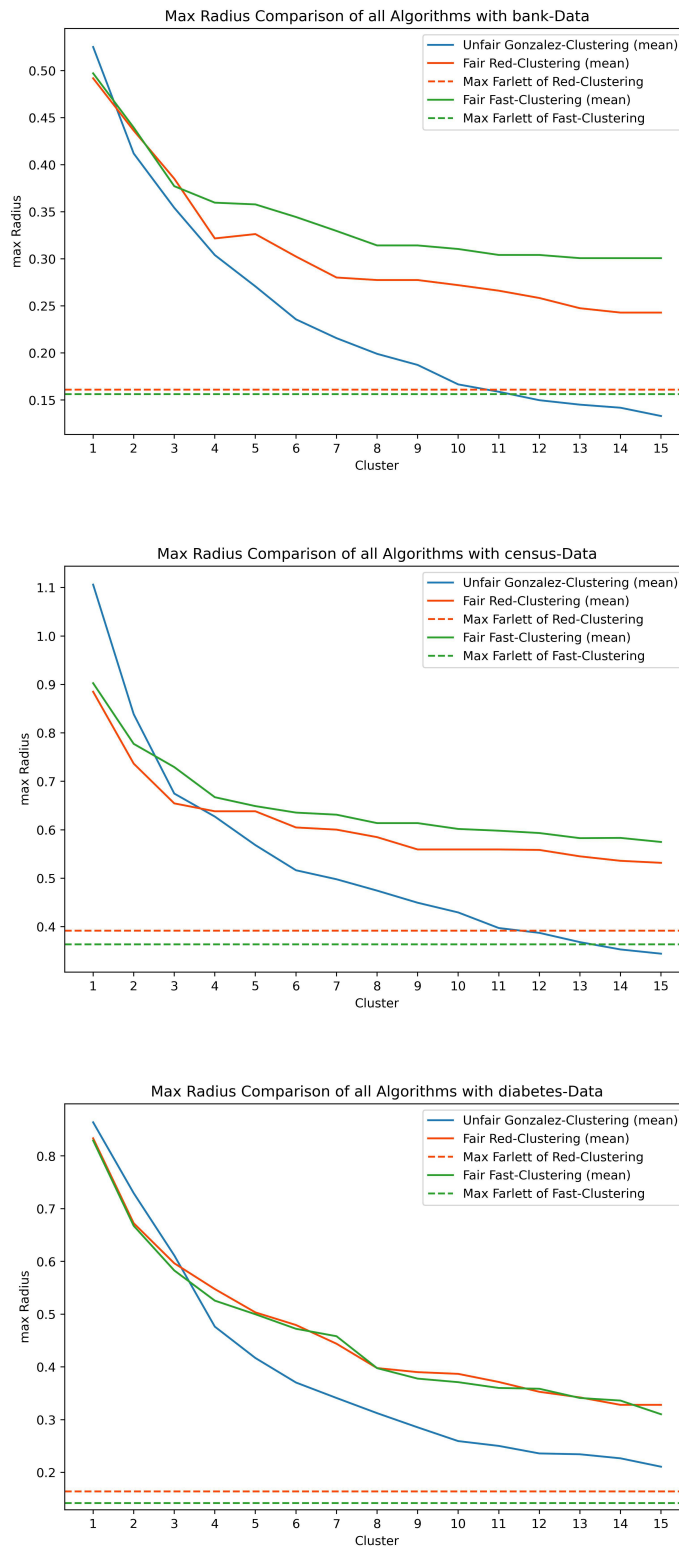


Abbildung 4.1: Zusammengefasste kosten der Cluster bis zu 15 Cluster

Kapitel 5

Auswertung

Was habe ich aus den Experimenten gelernt

5.1 Beschreibung der Bilder

5.2 Güte der Algorithmen

Konvergieren die Algorithmen gut? Wie groß ist der Kostenunterschied? Wie groß ist der Unterschied zwischen den Datensätzen?

5.3 Center-Aware Problem

Beim Fair-Anchor-Clustering 2.5.3 weisen wir explizit die Punkte, welche auch Zentren sind automatisch ihren eigenen Clustern zu. Approximativ wird die Güte dadurch nicht schlechter [Fas23] jedoch ist bei den Experimenten aufgefallen, dass bei spezifischen Datensamples der Maximale Radius des Clusters größer wird, obwohl wir mehr Zentren zulassen.

Durch weitere Nachforschung kam raus, dass es tatsächlich Randfälle gibt, in welchen der maximale Radius wieder ansteigen kann, wenn man ein neues Cluster dazu nimmt.

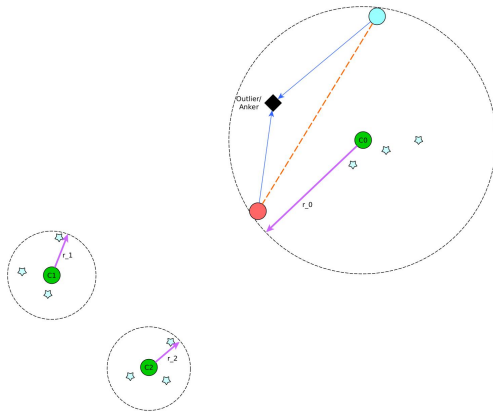


Abbildung 5.1: Clustering with $k = 3$

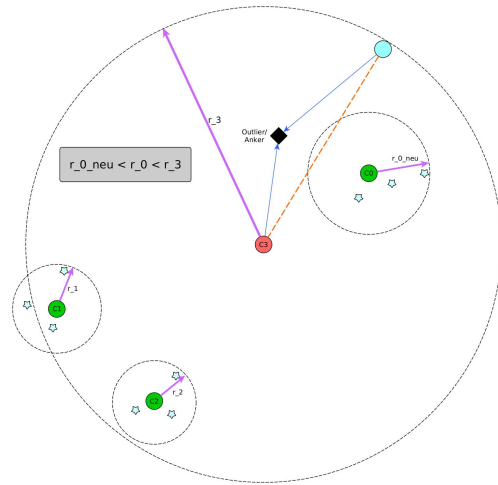


Abbildung 5.2: Clustering with $k = 4$

Beispiel einer Radiussteigerung

Hier ein Beispiel eines solchen Falles bei einer Steigerung von $k = 3$ auf $k = 4$. Voraussetzung dafür ist, dass ein Fairlettpaar $r \in P$ und $b \in P$ im Fall $k = 3$ Teil des Clusters c_0 ist. Im Beispielbild 5.1 ist zu sehen, dass r mit $\text{dist}(r, c_0)$ maßgeblich für den Maximalen Radius ist. Dabei sind r und b dem Cluster von c_0 zugewiesen, weil die Distanz des Ankers α zum Zentrum c_0 am kleinsten ist. Im nächsten Schritt wird r als neues Zentrum (c_4) gewählt, weil es der am weitesten entfernte Punkt von allen anderen Zentren ist. Allerdings muss jetzt sowohl r selbst als auch sein Fairlettpartner b dem neuen Cluster c_4 zugewiesen werden. Dadurch erhöht sich der maximale Radius enorm auf $\text{dist}(r, b)$.

Kapitel 6

Zitierungen

Einfach eine Datei, in der alles Zitiert wird, damit ich mir mein Literaturverzeichnis anschauen kann

Noch ein wenig Mehr Zitieren:

1. Chierichetti-Paper [Chi+18]
2. Zensusdaten [BK96]
3. Bankdaten [MRC12]
4. Diabetes (Falsch) [Kah]
5. Diabetes (Richtig) [Clo+14]
6. BA von Irina [Fas23]
7. Vorlesungsskript Clustering [Sch23]
8. Lemon Library [Lem]

Literatur

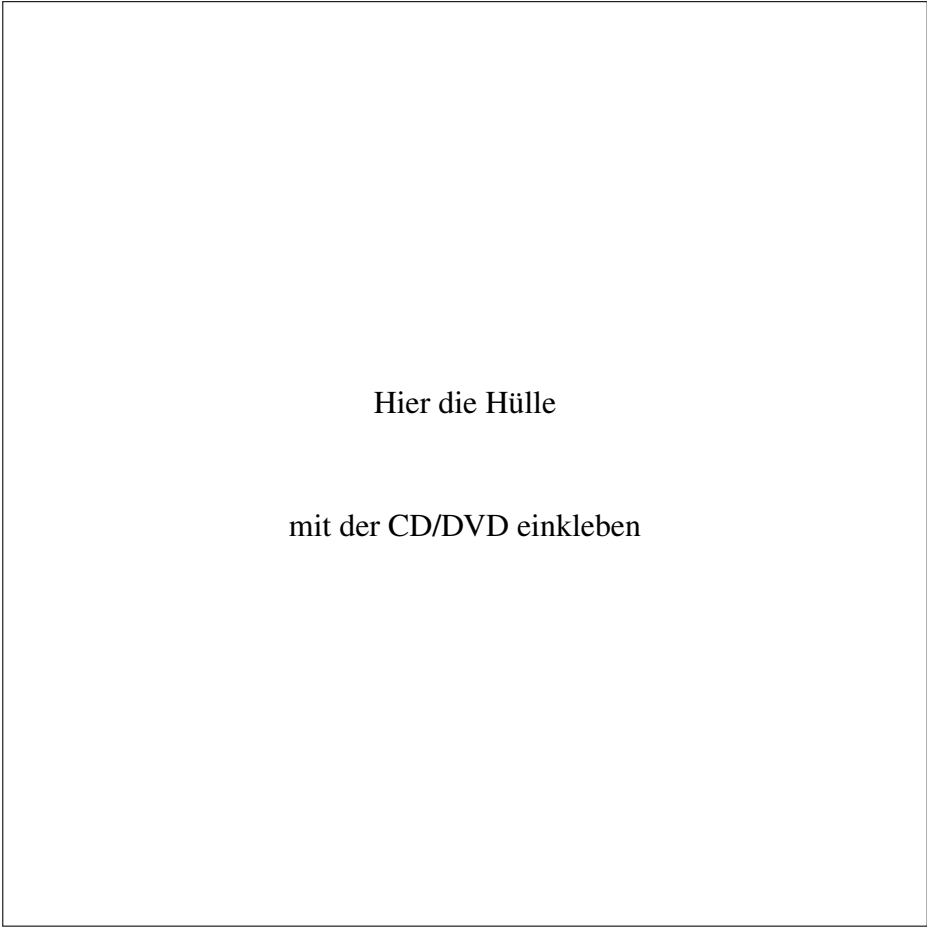
- [BK96] Barry Becker und Ronny Kohavi. *Adult*. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5XW20>. 1996.
- [Chi+18] Flavio Chierichetti u. a. *Fair Clustering Through Fairlets*. 2018. arXiv: 1802.05733 [cs.LG].
- [Clo+14] John Clore u. a. *Diabetes 130-US hospitals for years 1999-2008*. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5230J>. 2014.
- [Fas23] Irina Fast. „Fair k-Center Clustering mit Ausreißern“. Masterarbeit. Düsseldorf, NRW, Germany: Heinrich Heine University, Dez. 2023.
- [Gon85] Teofilo F. Gonzalez. „Clustering to minimize the maximum intercluster distance“. In: *Theoretical Computer Science* 38 (1985), S. 293–306. ISSN: 0304-3975. DOI: [https://doi.org/10.1016/0304-3975\(85\)90224-5](https://doi.org/10.1016/0304-3975(85)90224-5). URL: <https://www.sciencedirect.com/science/article/pii/0304397585902245>.
- [Kah] Michael Kahn. *Diabetes*. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5T59G>.
- [Lem] A Jüttner, B Dezsö und P Kovács. *Lemon Library - open source graph library*. <https://lemon.cs.elte.hu/trac/lemon>.
- [MRC12] S. Moro, P. Rita und P. Cortez. *Bank Marketing*. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5K306>. 2012.
- [Sch23] Melanie Schmidt. „Kombinatorische Algorithmen für Clusteringprobleme - Vorlesungsskript“. 2023.

Ehrenwörtliche Erklärung

Hiermit versichere ich, die vorliegende Bachelorarbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt zu haben. Alle Stellen, die aus den Quellen entnommen wurden, sind als solche kenntlich gemacht worden. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Düsseldorf, 29. Februar 2013

Carsten Krollmann



Hier die Hülle
mit der CD/DVD einkleben

Diese CD enthält:

- eine *pdf*-Version der vorliegenden Bachelorarbeit
- die $\text{\LaTeX}$ - und Grafik-Quelldateien der vorliegenden Bachelorarbeit samt aller verwendeten Skripte
- **[anpassen]** die Quelldateien der im Rahmen der Bachelorarbeit erstellten Software XYZ
- **[anpassen]** den zur Auswertung verwendeten Datensatz
- die Websites der verwendeten Internetquellen